



## SECURITY DOCUMENTATION

# Smart Contract Static Analysis Report

SprootFactory, SprootERC721 and SprootERC1155 collection contracts

REPORT DATE

13 Sep 2026

SOURCE COMMIT

[cc0aa4d](#)

COMPILER

solc 0.8.28

IN-SCOPE CODE

~700 lines

Full source: [github.com/sproot-art/contracts](https://github.com/sproot-art/contracts)

**Nature of this document.** This is an automated static analysis performed by the SPROOT team with open-source tools, with every result triaged by hand against the source. It is an **internal review, not an independent third-party audit**. Static analysis detects known vulnerability patterns; it does not prove the absence of defects.

## 1. Summary

Two industry-standard analyzers were run over the in-house contract source. No result identifies a way for any party to take funds or tokens from creators or collectors, to seize or upgrade a deployed collection, or to alter a collection's platform fee after deployment. Every result was reviewed individually; the dispositions are listed in section 4.

0

Results confirmed as exploitable

23

Slither results (102 detectors)

9

Aderyn results (0 high)

177

Unit, fuzz & invariant tests

## 2. Scope

| FILE                         | ROLE                                                | CODE LINES |
|------------------------------|-----------------------------------------------------|------------|
| src/SprootCollectionBase.sol | Shared mint accounting, phases, proceeds, royalties | 360        |
| src/SprootERC721.sol         | Sequential collection implementation                | 119        |
| src/SprootFactory.sol        | Deploys EIP-1167 clones; holds platform fees        | 77         |
| src/SprootERC1155.sol        | Edition collection implementation                   | 74         |
| src/SprootTypes.sol          | Structs, constants and custom errors                | 69         |
| src/ISprootFeeSink.sol       | Fee hand-off interface                              | 4          |

Out of scope: OpenZeppelin Contracts v5.1 and OpenZeppelin Contracts Upgradeable v5.1 (unmodified dependencies), tests, deployment scripts, and all off-chain components. Line counts exclude comments and blank lines. Build: optimizer enabled, 200 runs, EVM target **paris**.

## Deployed contracts (verified source)

| NETWORK                | CONTRACT                     | ADDRESS                                    |
|------------------------|------------------------------|--------------------------------------------|
| Ethereum (1)           | SprootFactory                | 0x0e72859Dd5A52e831EDD4c25f6dC506EE45675a5 |
| Ethereum (1)           | SprootERC721 implementation  | 0xA43a27f6fDb4b196Ce19322761Fa920CB76BE147 |
| Ethereum (1)           | SprootERC1155 implementation | 0xE79f93B63c2621d2237D315C036d195B9630D38A |
| Robinhood Chain (4663) | SprootFactory                | 0x1C62A0D9E959EdE2De4817A3b9a142aacA00A6b1 |
| Robinhood Chain (4663) | SprootERC721 implementation  | 0x5cC1941110a1356A9E06218BE35082c22c46A1d1 |
| Robinhood Chain (4663) | SprootERC1155 implementation | 0xbb4f4b61977296bec8253BbD9E19E3ef04E2553B |

Every creator collection is a 45-byte EIP-1167 minimal proxy that delegates to one of the implementations above, so its behaviour is exactly that implementation's source.

## 3. Methodology

| TOOL                           | VERSION     | CONFIGURATION                                                                                                      |
|--------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Slither</b> (Trail of Bits) | 0.11.6      | All 102 default detectors; dependency, test and script paths filtered; Foundry build of the source commit.         |
| <b>Aderyn</b> (Cyfrin)         | 0.6.8       | Default detector set over <code>src/</code> .                                                                      |
| <b>Foundry</b>                 | forge 1.5.1 | Unit, fuzz and invariant test suite covering access control, reentrancy, mint limits, phases and fee immutability. |

Each tool result was traced to its source location and classified as **False positive** (the pattern matched but the risk does not apply), **By design** (intended behaviour, with the rationale stated), or **Acknowledged** (code-quality item with no security impact).

## 4. Results and dispositions

### 4.1 Slither – 23 results

| SEVERITY | DETECTOR                 | LOCATION & ANALYSIS                                                                                                                                                                                                                  | DISPOSITION    |
|----------|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| High     | shadowing-state          | SprootERC721.__gap shares a name with SprootCollectionBase.__gap. Both are <b>private</b> storage-gap arrays with distinct slots; neither is read or written. No storage collision exists, and the contracts are not upgradeable.    | False positive |
| Medium   | incorrect-equality<br>x2 | amount == 0 in SprootCollectionBase.withdraw and SprootFactory.withdraw. The comparison only rejects an empty withdrawal. Forcibly sent ETH makes the balance non-zero, which is harmless; no invariant depends on an exact balance. | False positive |

| SEVERITY | DETECTOR                 | LOCATION & ANALYSIS                                                                                                                                                                                                                                                                                                                                                                                                         | DISPOSITION    |
|----------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| Low      | reentrancy-events ×5     | Events emitted after an external call in <code>_processMint</code> , <code>_processPhaseMint</code> , <code>_payPlatformFee</code> and both factory <code>create*Collection</code> functions. The callee is either the factory (fixed at initialization) or a freshly created clone's initializer. Mint entry points are <code>nonReentrant</code> and all state is written before the call. Only log ordering is affected. | False positive |
| Low      | timestamp ×4             | Mint windows and phase schedules compare against <code>block.timestamp</code> . A time-bounded sale requires a clock; validator influence is limited to seconds and does not affect price, supply or access.                                                                                                                                                                                                                | By design      |
| Info     | low-level-calls ×2       | <code>call{value}</code> in both <code>withdraw</code> functions. Deliberate: <code>transfer</code> 's 2,300-gas stipend breaks smart-contract wallets. Owner-only, <code>nonReentrant</code> , balance read before the call, and the return value is checked.                                                                                                                                                              | By design      |
| Info     | cyclomatic-complexity ×3 | <code>_setPhases</code> , <code>_processMint</code> , <code>_processPhaseMint</code> . Complexity reflects the number of explicit checks (supply, per-wallet, window, pause, allowlist, payment). Covered by dedicated tests.                                                                                                                                                                                               | Acknowledged   |
| Info     | naming-convention ×4     | <code>__gap</code> and <code>__SprootCollection_init</code> follow OpenZeppelin's upgradeable-contract naming convention.                                                                                                                                                                                                                                                                                                   | Acknowledged   |
| Info     | unused-state ×2          | <code>__gap</code> in <code>SprootERC721</code> and <code>SprootERC1155</code> are intentionally unused storage reservations.                                                                                                                                                                                                                                                                                               | By design      |

## 4.2 Aderyn – 0 high, 9 low

| ID  | DETECTOR                                     | ANALYSIS                                                                                                                                                                                                                                                                                                                                                    | DISPOSITION    |
|-----|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| L-1 | Centralization risk                          | Owner-only functions. On a collection the owner is the <i>creator</i> , set at initialization; SPROOT holds no role. The factory owner can only set the fee for <i>future</i> deploys (capped in bytecode at 0.002 ETH and bounded by the creator's signed maximum) and withdraw platform fees. No function touches a deployed collection's funds or logic. | By design      |
| L-2 | Costly operations inside loop                | Storage writes when pushing phases (bounded by <code>MAX_PHASES = 8</code> ) and per-token ERC-721 mints. Gas only.                                                                                                                                                                                                                                         | Acknowledged   |
| L-3 | <code>nonReentrant</code> not first modifier | On both <code>withdraw</code> functions it follows <code>onlyOwner</code> , a pure view check with no external call.                                                                                                                                                                                                                                        | False positive |
| L-4 | Loop contains revert                         | Phase validation must reject the whole schedule if any phase is invalid; partial acceptance would be unsafe.                                                                                                                                                                                                                                                | By design      |
| L-5 | State change without event                   | <code>setDefaultRoyalty</code> : owner-only, bounded at 10%; the value is always readable via ERC-2981 <code>royaltyInfo</code> .                                                                                                                                                                                                                           | Acknowledged   |
| L-6 | Unchecked return                             | <code>_registerToken</code> in <code>setupDrop</code> returns the new id, which is always the constant <code>DROP_ID = 1</code> (guarded by the preceding check).                                                                                                                                                                                           | False positive |
| L-7 | Uninitialized local variable                 | Loop counter <code>uint256 i</code> ; relies on Solidity's zero default.                                                                                                                                                                                                                                                                                    | False positive |

| ID  | DETECTOR              | ANALYSIS                                                                          | DISPOSITION  |
|-----|-----------------------|-----------------------------------------------------------------------------------|--------------|
| L-8 | Unused error          | <code>DropNotConfigured</code> is declared but not referenced. No runtime effect. | Acknowledged |
| L-9 | Unused state variable | Storage-gap arrays; same item as Slither <code>unused-state</code> .              | By design    |

## 5. Security properties of the design

These properties hold by construction and can be confirmed directly against the verified source:

- ✓ **No upgradeability.** No proxy admin, beacon or `upgradeTo`; deployed logic cannot change.
- ✓ **No platform authority.** The factory holds no admin, pause or mint role on any collection.
- ✓ **Exact payment.** Overpayment reverts; excess ETH is never retained.
- ✓ **Immutable fee.** Snapshotted at initialization with no setter; capped in bytecode; bounded by the creator's signed maximum.
- ✓ **On-chain enforcement.** Supply, per-wallet limits, windows, pause and allowlists are enforced by the contract.
- ✓ **Creator ownership.** The creator is `owner()` from initialization; two-step ownership transfer.
- ✓ **Reentrancy protection.** Checks-effects-interactions plus `ReentrancyGuard` on mint and withdraw.
- ✓ **Separated balances.** Fees are forwarded to the factory at mint; creator proceeds and platform fees never mix.
- ✓ **No token approvals requested.** No `setApprovalForAll`, `Permit`, `Permit2` or allowance flows.
- ✓ **Front-running resistant addresses.** `CREATE2` salts are bound to the deployer's address.

## 6. Limitations

Static analysis matches known vulnerability patterns and cannot establish that business logic is free of defects. This report was prepared by the SPROOT team and has not been reviewed by an independent security firm. It reflects the source published at [github.com/sproot-art/contracts@cc0aa4d](https://github.com/sproot-art/contracts@cc0aa4d); the dependencies listed as out of scope were not analysed.